Programming Assignment 2: MathMatrix

Due: Thursday, January 29th at 11pm CT

Learning Goals

- To design and implement a class using object-oriented programming principles
- To work with two-dimensional arrays to represent and manipulate data
- To practice designing robust test cases to ensure the correctness of your code

Assignment Guidelines and Specifications

This is an individual assignment, which means you cannot work with a partner on this assignment. All code submitted must be entirely your own.

For this assignment, there are some additional specifications:

- You may use any classes and methods from the Java standard library that you choose.
- **You must add conditional statements to check the preconditions for public methods.** If the preconditions are violated, throw an `IllegalArgumentException`.
    - Look at the starter code from Assignment 1 to see examples of how to do this.
- You may (and are encouraged to) create private helper methods to remove redundancy and provide structure to your solution. You may also (and are encouraged to) use public methods from the `MathMatrix` class when completing other methods in the class if this simplifies the solution.
- Do not add unnecessary instance variables. You should minimize redundant code.

Introduction

Mathematical matrices are used in a variety of fields, such as physics, engineering, probability and statistics, economics, biology, and computer science. They are also commonly used to solve systems of linear equations.

A matrix is simply a rectangular table of numbers, either integers or real numbers. Here is an example of a matrix:

$$\begin{bmatrix} 1 & 5 & 10 \\ 6 & 4 & 12 \end{bmatrix}$$

The above matrix has 2 rows and 3 columns. A matrix can have an equal number of rows and columns, also known as a square matrix, but it is not required to.

Getting Started

Here are the necessary files for this assignment:

- **`MathMatrix.java`**: This file contains the starter code for the MathMatrix class. You will need to complete this file according to the given specifications and submit this file.
- **`MathMatrixTester.java`**: This file contains the main method used for testing and the provided correctness tests. You will need to write your own tests in this file and submit this file.
- **`Stopwatch.java`**: This file contains the Stopwatch class, which you will need to use in your experiments. You will use this class to measure the time elapsed. You should not edit this code and do not need to submit this file.
- **`Stopwatch.html`**: This link contains the documentation for the Stopwatch class. You should reference this documentation to use the Stopwatch class in your experiments.

Overview

In this assignment, you will be designing and implementing a class that represents a mathematical matrix. Your matrices will only contain java `ints`.

You are required to use a native, two-dimensional array of `ints` as the underlying storage container for the matrix class, such as:

```
private int[][] values
```

You may choose the name of this container variable. However, we recommend that you name it 'values', 'nums', 'cells', or some other appropriate name. We advise **against** naming the container 'matrix' or 'mathMatrix'. This often causes confusion because it suggests that the array of `ints` *is* the `MathMatrix`, which is not true! Instead, this array of `ints` is simply the underlying storage container within the `MathMatrix`. If you are confused about this, please come to help hours so that we can help you understand before you get started on this assignment. Any lingering confusion about this will lead to larger issues later on in the assignment.

While you are writing the methods for the `MathMatrix` class, you'll notice that none of the methods alter the size of the matrix. This means that once a `MathMatrix` is created, its size can never be changed. Because of this, unlike the `IntList` we created in lecture, it does **not** make sense to have extra capacity in the underlying storage container. The size of the two-dimensional array of `ints` should instead be the exact **same** as the mathematical matrix it is representing. This also means that it is **not** necessary to track the number of rows and columns in the matrix with separate instance variables.

We will use 0-based indexing when referring to the rows and columns of the `MathMatrix`. This means that the first row and first column are at the respective 0th indices.

1. Constructors

In the `MathMatrix.java` file, implement the following constructor:

```
public MathMatrix(int numRows, int numCols, int initialVal)
```

In this first constructor, create a `MathMatrix` with `numRows` rows and `numCols` columns. All the elements of the matrix should equal `initialVal`. This method has the precondition that `numRows` and `numCols` must both be greater than zero.

In the `MathMatrix.java` file, also implement the following constructor:

```
public MathMatrix(int[][] mat)
```

In this constructor, create a `MathMatrix` with elements equal to the values in `mat`. Any changes made to `mat` after the constructor is run should not affect the MathMatrix, and any changes to the MathMatrix should not affect `mat`. In order to do this, you will need to make a **deep copy** of `mat`, as opposed to a shallow copy. Learn more about how to create a deep copy here. This method has the following preconditions: 1) `mat` should not be null, 2) the numbers of rows and columns in `mat` should be greater than 0, and 3) `mat` should be a rectangular matrix.

2. `getNumRows`

In the `MathMatrix.java` file, implement the method:

```
public int getNumRows()
```

This method should return the number of rows in this `MathMatrix`.

3. `getNumColumns`

In the `MathMatrix.java` file, implement the method:

```
public int getNumColumns()
```

This method should return the number of columns in this `MathMatrix`.

4. `getVal`

In the `MathMatrix.java` file, implement the method:

```
public int getVal(int row, int col)
```

This method should return the value in `MathMatrix` at the position specified by the `row` and `col` parameters. This method has the precondition that both the `row` and `col` parameters will be valid indices within the matrix dimensions.

5. add

Matrix addition is performed by adding the corresponding values of two matrices together. In order to perform this operation, the matrices must have the same dimensions.

The sum of two matrices *A* and *B*, denoted *A + B*, is a matrix that has the same number of rows and columns as *A* and *B*. *A + B* is computed by adding together the corresponding elements of *A* and *B*.

Below is a generalized example:

$$A + B = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \end{bmatrix} + \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \end{bmatrix} = \begin{bmatrix} a_{00} + b_{00} & a_{01} + b_{01} \\ a_{10} + b_{10} & a_{11} + b_{11} \\ a_{20} + b_{20} & a_{21} + b_{21} \end{bmatrix}$$

Here is an example using values in the matrices:

$$A + B = \begin{bmatrix} 1 & 5 \\ 17 & 3 \\ 5 & -3 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 5 & 2 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 + 5 & 5 + 6 \\ 17 + 5 & 3 + 2 \\ 5 + 2 & -3 + 1 \end{bmatrix} = \begin{bmatrix} 6 & 11 \\ 22 & 5 \\ 7 & -2 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public MathMatrix add(MathMatrix rightHandSide)
```

This method should return a **new** MathMatrix that is the result of adding the calling `MathMatrix` object and the `rightHandSide` `MathMatrix`. Neither the calling object nor `rightHandSide` should be altered in this method. As seen in the examples above, the dimensions of the new returned matrix are equal to the dimensions of the calling matrix. This method has the precondition that the `rightHandSide` matrix will have the same dimensions as the calling matrix.

6. subtract

Matrix subtraction is very similar to matrix addition. It is performed by subtracting the corresponding values of one matrix from the other. Again, in order to perform this operation, the matrices must have the same dimensions.

The difference of two matrices *A* and *B*, denoted *A - B*, is a matrix that has the same number of rows and columns as *A* and *B*. *A - B* is computed by subtracting the elements of *B* from the corresponding elements of *A*.

Below is a generalized example:

$$A - B = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \end{bmatrix} - \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \end{bmatrix} = \begin{bmatrix} a_{00} - b_{00} & a_{01} - b_{01} \\ a_{10} - b_{10} & a_{11} - b_{11} \\ a_{20} - b_{20} & a_{21} - b_{21} \end{bmatrix}$$

Here is an example using values in the matrices:

$$A - B = \begin{bmatrix} 1 & 5 \\ 17 & 3 \\ 5 & -3 \end{bmatrix} - \begin{bmatrix} 5 & 6 \\ 5 & 2 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 - 5 & 5 - 6 \\ 17 - 5 & 3 - 2 \\ 5 - 2 & -3 - 1 \end{bmatrix} = \begin{bmatrix} -4 & -1 \\ 12 & 1 \\ 3 & -4 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public MathMatrix subtract(MathMatrix rightHandSide)
```

This method should return a **new** MathMatrix that is the result of subtracting the `rightHandSide` `MathMatrix` from the calling `MathMatrix` object. Neither the calling object nor `rightHandSide` should be altered in this method. As seen in the examples above, the dimensions of the new returned matrix are equal to the dimensions of the calling matrix. This method has the precondition that the `rightHandSide` matrix will have the same dimensions as the calling matrix.

7. `multiply`

Matrix multiplication does **not** work like matrix addition or subtraction; it does **not** involve simply multiplying the corresponding values of the two matrices.

Unlike with matrix addition and subtraction, the multiplied matrices do not need to have the same exact dimensions. Instead, in order to perform matrix multiplication, the number of columns in the first matrix must be equal to the number of rows in the second matrix.

The result of two multiplied matrices *A* and *B*, denoted *AB*, is a matrix that has the same number of rows as the first matrix and the same number of columns as the second matrix. In other words, assume the matrix *A* has dimensions *M x N* and the matrix *B* has dimensions *N x P*. The matrix *AB* would have dimensions *M x P*. In *AB*, the value at the *i*th row and *j*th column is determined by multiplying term-by-term the values of the *i*th row of *A* and the *j*th column of *B*, and then summing these products.

For a more in-depth refresher, here is a video by Professor Mike Scott on matrix multiplication.

Below is a generalized example:

$$AB = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \\ a_{30} & a_{31} \end{bmatrix} \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \end{bmatrix} = \begin{bmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} & a_{00}b_{02} + a_{01}b_{12} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} & a_{10}b_{02} + a_{11}b_{12} \\ a_{20}b_{00} + a_{21}b_{10} & a_{20}b_{01} + a_{21}b_{11} & a_{20}b_{02} + a_{21}b_{12} \\ a_{30}b_{00} + a_{31}b_{10} & a_{30}b_{01} + a_{31}b_{11} & a_{30}b_{02} + a_{31}b_{12} \end{bmatrix}$$

Here is an example using values in the matrices:

$$AB = \begin{bmatrix} 2 & 4 \\ 5 & 2 \\ 1 & 5 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 3 & 4 & 1 \\ 1 & 8 & 4 \end{bmatrix} = \begin{bmatrix} 2 \cdot 3 + 4 \cdot 1 & 2 \cdot 4 + 4 \cdot 8 & 2 \cdot 1 + 4 \cdot 4 \\ 5 \cdot 3 + 2 \cdot 1 & 5 \cdot 4 + 2 \cdot 8 & 5 \cdot 1 + 2 \cdot 4 \\ 1 \cdot 3 + 5 \cdot 1 & 1 \cdot 4 + 5 \cdot 8 & 1 \cdot 1 + 5 \cdot 4 \\ 3 \cdot 3 + 6 \cdot 1 & 3 \cdot 4 + 6 \cdot 8 & 3 \cdot 1 + 6 \cdot 4 \end{bmatrix} = \begin{bmatrix} 10 & 40 & 18 \\ 17 & 36 & 13 \\ 8 & 44 & 21 \\ 15 & 60 & 27 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public MathMatrix multiply(MathMatrix rightHandSide)
```

This method should return a **new** MathMatrix that is the result of multiplying the calling `MathMatrix` object and the `rightHandSide` `MathMatrix`. Neither the calling object nor `rightHandSide` should be altered in this method. As seen in the examples above, the number of rows in the new returned matrix is equal to the number of rows in the calling `MathMatrix`. The number of columns in the new returned matrix is equal to the number of columns in the `rightHandSide` `MathMatrix`. This method has the precondition that the number of columns in the calling matrix is equal to the number of rows in the `rightHandSide` matrix.

8. `getScaledMatrix`

Scalar multiplication is performed by multiplying every value in a matrix by some fixed constant, the scale factor. Below is a generalized example where *F* is the scale factor:

$$F \cdot A = F \cdot \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \end{bmatrix} = \begin{bmatrix} F \cdot a_{00} & F \cdot a_{01} \\ F \cdot a_{10} & F \cdot a_{11} \\ F \cdot a_{20} & F \cdot a_{21} \end{bmatrix}$$

Here is an example using values in the matrices, where the scale factor is 2:

$$2 \cdot A = 2 \cdot \begin{bmatrix} 3 & 4 \\ 6 & 2 \\ 9 & 10 \end{bmatrix} = \begin{bmatrix} 2 \cdot 3 & 2 \cdot 4 \\ 2 \cdot 6 & 2 \cdot 2 \\ 2 \cdot 9 & 2 \cdot 10 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 12 & 4 \\ 18 & 20 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public MathMatrix getScaledMatrix(int factor)
```

This method should return a **new** MathMatrix that is the result of scaling the calling matrix by `factor`. The calling object should not be altered in this method. As seen in the examples above, the dimensions of the new returned matrix are equal to the dimensions of the calling matrix. This method has no preconditions.

9. `getTranspose`

To transpose a matrix is to switch the row and column indices of a matrix *A* to produce another matrix, denoted A^T. The *N*th row of matrix *A* becomes the *N*th column of the resulting matrix A^T. Below is a generalized example:

$$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \end{bmatrix}, \; A^T = \begin{bmatrix} a_{00} & a_{10} & a_{20} \\ a_{01} & a_{11} & a_{21} \end{bmatrix}$$

Here is an example using values in the matrices:

$$A = \begin{bmatrix} 2 & 5 \\ 4 & 2 \\ 0 & 12 \end{bmatrix}, \; A^T = \begin{bmatrix} 2 & 4 & 0 \\ 5 & 2 & 12 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public MathMatrix getTranspose()
```

This method should return a **new** MathMatrix that is the result of transposing the calling matrix. The calling object should not be altered in this method. This method has no preconditions.

10. `equals`

In the `MathMatrix.java` file, implement the method:

```
public boolean equals(Object rightHandSide)
```

This method should return true if the `rightHandSide` object is the same size as the calling `MathMatrix` object and if all the values in the two `MathMatrix` objects are the same. Otherwise, this method should return false. This method has no preconditions.

In the method, there is already some starter code that checks if `rightHandSide` is a non-null `MathMatrix` object. If it is not, the method immediately returns false. If it is, `rightHandSide` is cast to a `MathMatrix` object called `otherMathMatrix`. We will learn more about this code in an upcoming lecture. For now, what this means is that:

- All of your code (that checks if the size of the two matrices are the same and checks if all the values in the two matrices are the same) should be written after this starter code. You can alter the '`return True`' statement at the end of the method as necessary. This is only currently in the starter code to avoid compile errors.
- You should only be using `otherMathMatrix` in the code you write and should not be using `rightHandSide`.
- You can access the private instance variables and the methods of `otherMathMatrix` in this method.

11. `toString`

In the `MathMatrix.java` file, implement the method:

```
public String toString()
```

This method should return a String representation of the matrix.

The String representation of the matrix must strictly follow these specifications:

- The order of the values in the String representation should correspond to the order of the elements in the matrix.
- Every row should begin and end with a vertical bar ("|").
- Use newline characters ("\n") to create the line breaks between the rows of the matrix. Every row should end with a newline character, even the last row of the matrix.
- All "cell" entries should be right-justified.
- The space for every "cell" of the String representation is equal to the longest value in the matrix plus 1. You will need to determine the length of each `int` in the matrix to find the longest one (*Note*: don't forget to account for negative signs when determining the length). *Hint*: find the length of an `int` by converting it to a `String` and finding the length of the String. Here is some sample code:

```java
int x = 78712;

// Option 1
String s = "" + x;
int lengthOfInt = s.length();

// Option 2
int lengthOfX = ("" + x).length();
```

For example, given the following `MathMatrix`:

$$\begin{bmatrix} 10 & 100 & 101 & -1000 \\ 1000 & 10 & 55 & 4 \\ 1 & -1 & 4 & 0 \end{bmatrix}$$

You should return the following `String` representation:

```
|    10   100   101 -1000|
|  1000    10    55     4|
|     1    -1     4     0|
```

In the example above, it can be difficult to determine how many spaces there are between numbers. In the following example, the spaces have been replaced by periods in order to make the amount of spaces clearer. However, your final output should not include periods.

```
|....10...100...101.-1000|
|..1000....10....55.....4|
|.....1....-1.....4.....0|
```

When implementing this method, you may optionally use the format method and formatting string syntax. Here is an additional introduction to formatting String syntax. You are not required to use the format method if you do not wish to.

12. `isUpperTriangular`

A matrix is upper triangular if it is a square matrix and all the values below the main diagonal are zero. The main diagonal is made up of the cells whose row and column values are equal (the diagonal from the top left corner to the bottom right corner).

Below is a general upper triangular matrix, *U*:

$$U = \begin{bmatrix} u_{00} & u_{01} & u_{02} & u_{03} \\ 0 & u_{11} & u_{12} & u_{13} \\ 0 & 0 & u_{22} & u_{23} \\ 0 & 0 & 0 & u_{33} \end{bmatrix}$$

The main diagonal of *U* is made up of the cells u_{00}, u_{11}, u_{22}, and u_{33}. All of the values under this diagonal are zero, which therefore makes *U* an upper diagonal matrix. Any 1 x 1 matrix is an upper diagonal matrix. Here are some examples of upper diagonal matrices with values:

$$[0], \quad [1], \quad \begin{bmatrix} 5 & 4 \\ 0 & 2 \end{bmatrix}, \quad \begin{bmatrix} 1 & 4 & 1 \\ 0 & 6 & 4 \\ 0 & 0 & 1 \end{bmatrix}$$

In the `MathMatrix.java` file, implement the method:

```
public boolean isUpperTriangular()
```

This method should return true if the calling `MathMatrix` object is upper triangular. Otherwise, this method should return false. This method has the precondition that the matrix must be square, i.e., the matrix should have an equal number of rows and columns.

Experiments

Now it's time to use your completed class to perform some experiments! You will use the Stopwatch class to record the time it takes to perform various operations on `MathMatrix` objects. The Stopwatch class has methods that return the elapsed time in seconds or nanoseconds between starting and stopping the Stopwatch. Below is some example code on how to use the Stopwatch class. Reference the Stopwatch class documentation for more details.

```
Stopwatch s = new Stopwatch();
s.start();
//code to time
s.stop();
```

Write your results and answers in a comment at the top of the `MathMatrixTester.java` file. Include the code to conduct the experiments in the `MathMatrixTester.java` file, but before submitting, make sure to comment out all experiment code (especially any code that utilizes the Stopwatch class).

Note: You may not share your experiment code with other students. This is still code that you are expected to write individually and still follows the academic integrity guidelines set forth.

Experiment 1: `add`

1. Create two MathMatrix objects, each with 500 rows and 500 columns. Fill both matrices with random integers.
2. Use the Stopwatch class to record the time it takes to add these two matrices together (using your `add` method). When doing this experiment, you should not measure the amount of time it takes to create the matrix objects, only the amount of time it takes to **add** the two matrices together.
3. Repeat this experiment 1000 times and output the **total** amount of time it takes to do all 1000 experiments. Do not create new `MathMatrix` objects for each test; instead simply reuse the two matrix objects you created for the first test on the other 999 tests.
4. Adjust the number of rows and columns in the two matrices so that the total amount of time for the 1000 tests is **at least one second**. As you adjust the dimensions of the matrices, make sure that both matrices remain square and have the same dimensions.
5. Record the dimensions of the matrices and the total amount of time it took for 1000 `add` tests in `MathMatrixTester.java`.
6. Create two new MathMatrix objects, with double the number of rows and columns as the matrix in the previous step, and fill both matrices with random integers. For example, let's say you found in the previous step that you needed a matrix with 800 rows and 800 columns to get a total time of at least one second. In this step, you would create two matrices with 1600 rows and 1600 columns.
7. Use the Stopwatch class to record the total time it takes to add these two larger matrices together 1000 times. Again, when doing this experiment, you should not measure the amount of time it takes to create the matrix objects, only the amount of time it takes to **add** the two matrices together. Do not create new `MathMatrix` objects for each test; instead, reuse the two matrix objects for all tests.
8. Record the dimensions of these larger matrices and the total amount of time it took for 1000 `add` tests on the larger matrices in `MathMatrixTester.java`.
9. Repeat steps 6 through 9 one more time. You will again double the dimensions of the matrices, conduct 1000 `add` experiments, determine the total amount of time, and record these results.
10. Note that during these experiments, you may get an "out of heap space" error. If this happens, increase the size of the heap. All of the possible command line flags are on this page. In Eclipse, you can set a command line flag for your program. Follow the instructions for enabling assertions and include the flag -Xmx*size* where *size* is the new requested heap size. For example, to increase the heap size to 120 mb, include the command line flag -Xmx120m.

Experiment 2: `multiply`

1. Repeat Experiment 1 for the multiply method. Only repeat each set of experiments 100 times, rather than 1000 times. Also, in Step 4, adjust the starting number of rows and columns so that the total amount of time for the 100 tests is **just over one second**.

Questions

1. Based on the results of Experiment 1, how long do you expect the `add` method to take if you doubled the dimensions of the MathMatrix objects again?
2. What is the Big O of the `add` operation given two N by N matrices based on an analysis of your code? Does your timing data support this?
3. Based on the results of Experiment 2, how long do you expect the `multiply` method to take if you doubled the dimensions of the MathMatrix objects again?
4. What is the Big O of the `multiply` operation given two N by N matrices based on analysis of your code? Does your timing data support this?
5. How large a matrix can you create before your program runs out of heap memory (assuming you are using the default heap size and not using any command line flags to increase the heap size)? In other words, what size matrix causes a Java OutOfMemoryError?
6. Based on the largest matrix that you were able to create successfully in Question 5, estimate the amount of memory your program is allocated. State your answer in megabytes. What percentage of your computer's RAM did your program use before crashing? *Hint: recall that an int in Java requires 4 bytes.*

Note: You may find that for your `add` and `multiply` methods, the Big O values when analyzing the code and when analyzing the experimental timing data do not match. That is perfectly ok and normal! When answering the questions, be honest about what your data shows; do not change your data to match your expectations. We are not grading based on the "correctness" of your experimental timing data. If you are curious about why the experimental timing data does not seem to reflect the Big O of the methods, come to help hours to chat about it (and learn about this thing called caching)!

Testing

You are required to write and submit your own test cases in `MathMatrixTester.java`. You are expected to write:

- 2 new test cases for `getNumRows`
- 2 new test cases for `getNumColumns`
- 2 new test cases for `getVal`
- 2 new test cases for `add`
- 2 new test cases for `subtract`
- 2 new test cases for `multiply`
- 2 new test cases for `getScaledMatrix`
- 2 new test cases for `getTranspose`
- 2 new test cases for `equals`
- 2 new test cases for `toString`
- 2 new test cases for `isUpperTriangular`

On the last assignment, we shared with you a few notes about writing test cases. Those notes are copied here for your convenience:

- You may copy the structure of the provided tests. You simply need to change the data and the expected results to reflect your new cases.
- Any test cases that you write should not violate the preconditions of the method.
- You must group the test cases by method and comment which method the test case is for.
- You are not required to write test cases for any private helper methods.
- **Delete the test cases that we have provided for you.**

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Implement all the required methods?
2. Conduct and record your answers for the experiments? Include the experiment code in `MathMatrixTester.java`?
3. Check the preconditions in public methods and throw exceptions as necessary?
4. Ensure your program does not suffer from a compile error or runtime error?
5. Fill in the header comments in both files?
6. Follow the program hygiene guide?
7. Follow the assignment specifications from page 3 of the Assignment handout?
8. Ensure your program passes the given tests? Note: passing the provided tests does not mean your solution is correct. We will use hidden tests to test edge cases.
9. Create and add the required number of additional test cases?
10. Delete the given test cases?

If you have done all these things, then you're ready to submit `MathMatrix.java` and `MathMatrixTester.java` to Gradescope!

Thank you to Professor Mike Scott for creating the original version of this assignment!
Many of the descriptions of the matrix operations are from Wikipedia.